AKTU

B.Tech 5th Sem

GW
GATEWAY CLASSES

CS IT & CS Allied

Object Oriented System Design with C++ (BCS054)

GATEWAY CLASSES

Notes

as per AKTU Syllabus

Unit-3

By Amol Sharma sir

- Pursuing Ph.D. from IIT (BHU)
- Wipro Certified Faculty (WCF)

Object-Oriented Analysis and Design (OOAD)

"Object-oriented analysis and design (OOAD) is a software engineering approach that models a system as a group of interacting objects."

Object-oriented analysis (OOA) applies object-modeling techniques to analyze the requirements for a system. It aims at *understanding, finding and describing concepts in the problem domain.*
Object-oriented design (OOD) elaborates the analysis models to produce implementation specifications. It is meant for *understanding and defining software solution/objects that represent the analysis concepts and will eventually be implemented in code.*

OOA focuses on what the system does, OOD on how the system does it.

- Each object represents some entity of interest in the system being modeled, and is characterized by its identity, its state (attributes and data elements), and its behavior.
- A set of similar objects are represented through their common class.
- Various models can be created to show the static structure, dynamic behavior, and run-time deployment of these collaborating objects.
- There are a number of different notations for representing these models, such as the Object Modeling Technique (OMT), Unified Modeling Language (UML).

Object Modeling Technique (OMT)

Object Modeling Technique (OMT) is real world based object modeling approach for software modeling and designing. It was developed basically as a method to develop object-oriented systems and to support object-oriented programming. It describes the static structure of the system.
OMT was developed around 1991 by Rumbaugh, Blaha, Premerlani, Eddy and Lorensen. OMT is a predecessor of the Unified Modeling Language (UML). Many OMT modeling elements are common to UML.

GATEWAY CLASSES

Purpose of Object Modeling Technique:
- To test physical entity before construction of them.
- To make communication easier with the customers.
- To present information in an alternative way i.e. visualization.
- To reduce the complexity of software.
- To solve the real world problems.

Object Modeling Technique's Models:

There are three main types of models that have been proposed by OMT:

1. **Object Model:** The object model represents the static and most stable phenomena in the modeled domain. Main concepts are classes and associations with attributes and operations. Aggregation and generalization (with multiple inheritances) are predefined relationships.

2. **Dynamic Model:** The dynamic model represents a state/transition view on the model. Main concepts are states, transitions between states, and events to trigger transitions. Actions can be modeled as occurring within states. Generalization and aggregation (concurrency) are predefined relationships

3. **Functional Model:** The functional model handles the process perspective of the model, corresponding roughly to data flow diagrams. Functional Model focuses on the how data is flowing, where data is stored and different processes. Main concepts involved in Functional Model are data, data flow, data store, process and actors.

Phases of Object Modeling Technique:

OMT has the following phases:

1. **Analysis:**
   The goal of analysis is to develop a model of what the system will do. The model is expressed in terms of objects and relationships, dynamic control flow, and functional transformations. The process of capturing requirements and consulting with the requestor should continue throughout analysis. Analysis phase starts with setting a goal i.e. finding the problem statement. Problem statement is further divided into above discussed three models i.e. object, dynamic and functional model.

2. **System Design:**
   This is the second phase of the object modeling technique and it comes after the analysis phase. It determines all system architecture, concurrent tasks and data storage. High level architecture of the system is designed during this phase.

3. **Object Design:**
   Object design is the third phase of the object modeling technique and after system design is over, this phase comes. During Object Design we elaborate the analysis model and provide a detailed basis for implementation. We make the decisions that are necessary to realize a system without descending into the particular details of an individual language or database system. Object design starts a shift away from the real world orientation of the analysis model towards the computer orientation required for practical implementation.

4. **Implementation:**
   This is the last phase of the object modeling technique. It is all about converting prepared design into the software. Design is translated into the implementation in this phase.

GATEWAY CLASSES

(*)

Object Design :- 'Object design' is a process of refinement or adding details;

During object design, the designer takes the strategy chosen in the earlier phases and then refines it and adds necessary details to it. In this phase, major emphasis is on computer concepts such as minimizing execution time, memory and other measures of cost.

The object design phase not only determines the full definitions of classes, associations, interfaces and algorithms of the methods but also adds internal objects (not identified in the previous phases) and optimizes data structures & algorithms.

Steps in Object Design :- Object design is an iterative process which the object design is complete at one level of abstraction, more details can be added and the design can be taken to a finer level of detail and this may include identifying new attributes and classes operations of the classes or even new classes revising or obtaining relationships among objects etc. Thus, during object design, the designer must perform the following steps :-

GATEWAY CLASSES

① Combine the three models to obtain operations on classes
② Design algorithms to implement operations.
③ Optimization
④ Implementation of control
⑤ Increase Inheritance
⑥ Design associations
⑦ Determine object representation
⑧ Group associations and classes into packages (modules).

① Combining the three models :- After analysis, we have three models, the Object model, dynamic model and functional model on which the design has to be constructed. All the three models need to be combined for laying out an appropriate design. These models may be understood as follows :-

(i) Object Model :- The object model describes the classes of objects in the system, including their attributes and the operations that they support.

(ii) Dynamic Model :- The dynamic model describes how the system responds to external events. This model specifies the control structure or the flow of control for a program.

(iii) Functional Model :- The functional model describes the operations that the system must implement.

The Object Model is the main framework around which design is constructed. The designer must convert

the actions & activities initiated as a result of transitions in a state diagram of a dynamic model

and

the processes of the functional model, that are the expansion of actions in dynamic model in the form of a data flow diagram

into

operations attached to classes in the object model

As a result of combining the three models the logical structure of analysis begins to be converted into the physical organization of a program

② Designing Algorithms :- Each operation specified in the functional model must be formulated as an algorithm because the algorithm shows 'how' the operation is to be done.

An algorithm designer must do the following activities :-

(i) Choose algorithms :- An algorithm is chosen that minimizes the cost of implementation of an operation.

There may be several alternative algorithms for an operation. So while choosing among alternatives following factors must be considered:-

a) Computational complexity, increase in processor time as a function of input size of the data structures.

b) Ease of implementation & understandability.

c) Flexibility in extending or modifying algorithm

d) Fine tuning the object model, if the object model were structured differently, would there be other alternatives?

(ii) Choosing Data Structures :- During Object design form of data structure is choosen that permits efficient algorithms. Data structures organize data in a form that is convenient for an algorithm that use it. Such data structures include arrays, lists, queues, priority queues, trees, sets etc.

(iii) Defining Internal Classes and Operations :- A complex operations can be defined in terms of lower-level operations on simpler objects. Sometimes there may be a need to add new classes that were not mentioned directly in the client's description of the problem. Such internal classes and operations are also identified during object design

(iv) Assigning Responsibility for operations :- Some times operations does not belong to single class of objects. Assigning responsibility for such operations to classes is complex activity that must be performed very carefully in the object design phase itself.

③ Design Optimization :- The basic design model uses the analysis model as the framework for implementation. The inefficient but semantically correct analysis model can be optimized to make the implementation more efficient but as an optimized system is more obscure and less likely to be reusable in another context thus while optimization is being done, the designer must maintain balance between efficiency and clarity. During design optimization, the designer must:

① Add redundant associations to minimize access cost and maximize convenience.

② Algorithm Optimization, rearranging the execution order or computation for greater efficiency.

③ Save derived attributes, Data that is derived from other data can be 'cached' or stored in its computed form to avoid the overhead of recomputing it.

Example :- One key to algorithm optimization is to

eliminate dead paths as early as possible. Suppose we want to find employees in a company who speak both Hindi and English provided that 5 employees speak English and 50 speak Hindi. So it is better to test the 5 English speakers first that they can speak Hindi or not. In this way the search is narrowed.

④ Implementation of control :- The control structure of a system is represented by the dynamic model. Thus implementation of control is just the realisation of dynamic model. In the object design, the designer must refine the strategy for implementing the state-event models present in the dynamic model.

There are three basic approaches to implementing the dynamic model :-

(i) ① State as location with in a program (procedure driven system) :- The location of control with in a program implicitly defines the program state. Any finite state machine can easily be implemented using program constructs like statements, conditionals, loops etc.

(ii) The technique of converting a state diagram or more over an activity diagram to code is as follows :

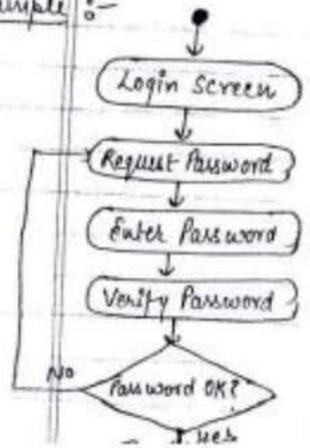
① Identify the main control path from initial state to the ending state corresponding to the normally expected sequence of events. Write the name of the activity states along this path and that becomes a sequence of statements in the program.

② Alternate paths that branch off the main path and rejoin it later becomes conditional statements.

③ Backward paths that branch off the main path and rejoin it earlier become loops.

④ The states that correspond to exception conditions must be handled using error ~~subroutines~~ subroutines status flags or exception handling.

Example :-



⇒

Pseudocode

```
do
{
    ask for password
    read password
    verify password
} while (password not OK)
```

GATEHUB CLASSES

(ii) State Machine Engine :- The most direct approach to implementing control is to have some way of explicitly representing and executing state machines. This approach allows to quickly obtain the skeleton prototype of the system through which it can be verified that the basic flow of control is correct or not.

Example :- A parser, such as unix yacc or lex, produces an explicit state machine to implement a user interface.

(iii) Control as concurrent task :- Control is implemented as concurrent tasks so that the inherent concurrency of real objects is maintained.

Example : Java supports concurrency through Multithreading.

(5) Adjustment of Inheritance :- Inheritance is a mechanism for implementing generalization in which the behavior of a superclass is shared by all its subclasses. Inheritance offers some really exciting & direct advantages such as

a) Code sharing & reuse
b) Modularity
c) Improves extensibility
d) Improves configuration management etc.

Thus, classes and operations must be adjusted to increase the amount of inheritance. For this the designer should do the following activities :-

(i) Rearranging classes and operations :- Some times by slightly modifying the classes and operations, the operation can be made to be the part of a common superclass from which it can be inherited by the subclasses.

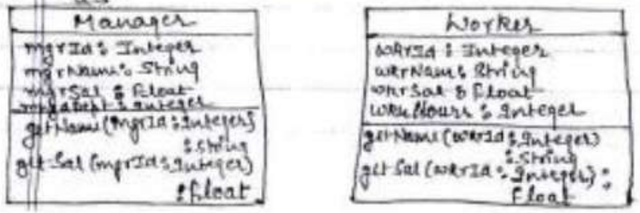
The following kind of adjustments can be used to increase inheritance :-

a) Missing arguements can be added to operations and ignored wherever not applicable.

b) Give the same name to the attributes accross the classes and move them to a superclass.

c) If operation may be defined for most of the classes but don't mean for some classes then this operation must acts as a no-op for those classes that don't care about it and on those classes it must have no effect.

Example :- The two classes manager & worker states as



GATEWAY CLASSES

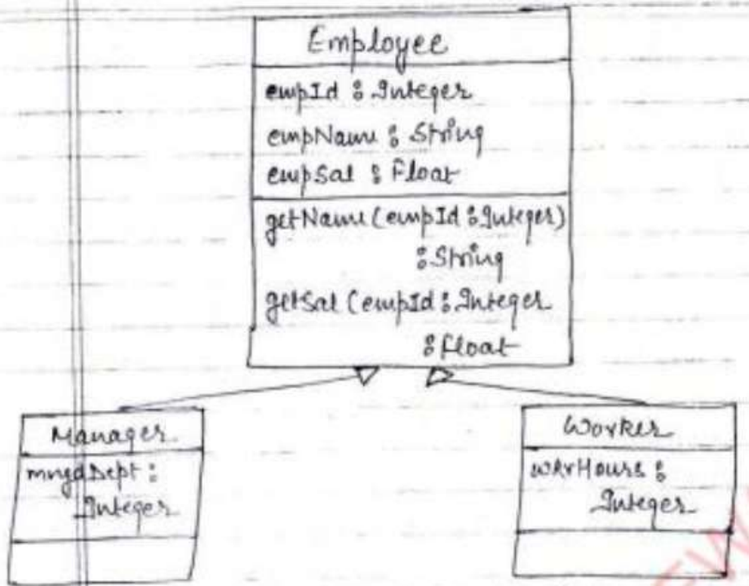


Fig: Manager & Worker Classes Inherit Employee Class (After making the adjustments)

(ii) Abstracting out common behavior :- It is possible that the two classes are really specialized variations of the same thing when viewed at a higher level of abstraction. Thus common behavior must be recognized and after it a common super class can be created that implements the shared features leaving only the specialized features in the sub-classes. This transformation of object model is called abstracting out common behavior. Usually the resulting class is an abstract class that donot have direct instances but its subclasses surely do.

(iii) Use 'delegation' to share implementation :- Though inheritance is advantageous but it must not be forcefully introduced when there is no scope for it.

Example :- If we try to inherit 'Stack' from a 'List', then addition can be done at arbitrary positions in the list but in stack insertion & deletion always take place from the same end. Thus if accidentally the insert at arbitrary position is applied to stack then it will lost its basic & intended nature and that is really problematic.

There is other technique, to achieve the same goal as inheritance, namely delegation. Delegation involves selective invocation of desired functions of a class by the object of another class. Only meaningful operations are delegated to the second object. Thus there is no danger of inheriting meaningless operations by accident.

GATEWAY CLASSES

(c)

Object Representation :- The attributes of classes are finally implemented in terms of built in data types (primitive) such as integers, floats, strings etc.

Thus designer must choose when to

a) Use primitive types in representing objects

b) Combine groups of related objects.

Example

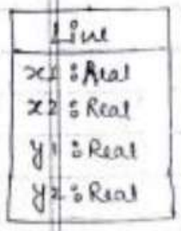


Fig: Object as primitive types

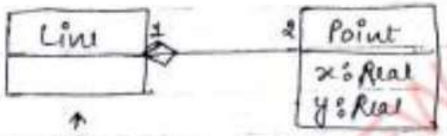


Fig: Group of objects

(7) **Physical Packaging** :- In any large project, careful partioning of implementation units into packages is important to permit different persons to co-operatively work on a program. Packaging involves the following issues:-

(i) **Information Hiding** :- The classes must be treated as "blackboxes" whose external interface is public but whose internal details are hidden.

For information hiding the object designer must do the following :-

(i) Internal information of a class must be hided so that implementation of a class may be changed without any change in the code of its clients.

(ii) Public and private interfaces of classes and packages must be carefully and well defined.

(iii) Keep a fine balance between information hiding and optimization as information hiding seeks to minimize dependencies where as optimization increases dependency in the form of redundant components and associations.

(iv) Try to limit the scope of methods.

(ii) **Coherence of entities** :- The design principle of coherence of entities says,

"An entity such as a class, package or module is coherent if is organized and all its parts are closely related and fit together towards a common goal."

Thus the entities must be highly cohesive and loosely coupled.

GATEWAY CLASSES

Coherence of entities may be ensured if,

(A) Policy & Implementation methods must be seperated. Policy methods normally make context dependent decisions and contain calls to Implementation methods. Implementation methods are the execution of fully specified algorithms. These methods are complicated but are not context dependent. Implementation methods are highly reusable due to this seperation.

(B) If a class is too complicated it must be broken up using generalisation or aggregation.

(iii) Constructing Modules :- Modules should be defined so that their Interfaces are minimal & well defined. The Interface between two modules contain association among classes and operations causing access across modules.

→ The classes that are closely connected by associations should be in the same module.

→ Modules should have functional cohesiveness or unity of purpose.

→ The classes in the module must be closely related representing similar things.

→ Try to minimise coupling between two modules.

In analysis the object model is partioned into modules. During design new classes are added that may fit in the existing modules or if required new modules may be introduced.

(8) Documenting Design Decisions :- Documentation is the best way of transmitting the design to others and recording it for future reference upto maintenance. Thus the design decisions taken in the previous steps must be documented.

⇒ The design document include a revised and much more detailed description of the object model in both graphical & textual form.

⇒ The design document specify the operations of functional model by giving their arguments, results, I/O mappings, algorithms & side effects.

⇒ Structured pseudocodes obtained by the implementation of control specified by dynamic model are also documented.

⇒ The design document contains many optimization & implementation artifacts.

⇒ Design document must have clear & user oriented description of the system that can be used for validation & reference for maintenance.

⇒ An analysis element should be easily traced in the design document.

GATE IT CLASSES

STRUCTURED ANALYSIS/ STRUCTURED DESIGN (SA/SD)

Structured Analysis/Structured Design (SA/SD) is a representative of the data flow approach and is pervasive, applicable to many problems, and well documented.

SA/SD largely uses ***functional decomposition*** in which each function is analyzed and hierarchically decomposed into more detailed functions. Simultaneously, high level data is also decomposed into more detailed data. SA/SD technique results in high-level design. This is also known as ***top-down decomposition.***

Structured Analysis

In structured analysis, textual problem description is converted into a graphical model. It is done using ***Data Flow Diagrams (DFDs)***. DFD graphically represents the results of structured analysis. DFD is a hierarchical graphical model that shows the different functions (or processes) of the system and data interchange among these processes.

The results of structured analysis can be easily understood even by ordinary customers as it does not require computer knowledge, directly represents customer's perception of the problem and uses customer's terminology for naming different functions and data. Results of structured analysis may be reviewed by customers to check whether it captures all their requirements.

Structured Design

In structured design, the functions represented in the DFD are mapped to a ***module structure***. Module structure may also be called as ***software architecture***. The aim of structured design is to transform the results of structured analysis (DFD representation) into a structure chart. A ***structure chart*** represents the software architecture consisting of various ***modules*** making up the system, ***module dependency*** (i.e. which module calls which other modules), and ***parameters*** passed among different modules.

"Purpose of structured analysis is to capture the detailed structure of the system as the user views it whereas the purpose of structured design is to arrive at a form that is suitable for implementation in some programming language."

The ***OMT and SA/SD methodologies*** both incorporate similar modeling components. Both methodologies support three orthogonal views of a system – the object, dynamic and functional models. The OMT and SAJSD methodologies differ in the relative emphasis that they place on the various modeling components. OMT designs are dominated by the object model. The real world paradigm of objects and relationships provides the context for understanding dynamic and functional behavior. In contrast, SA/SD stresses functional decomposition. A system is viewed primarily as providing one or more functions to the end user. In the SA/SD approach, the

HIGHWAY CLASSES

functional model dominates, the dynamic model is next most important, and the object model least important. In contrast, OMT modeling regards the object model as most important, then the dynamic model, and finally the functional model.

Summary of SA/SD Approach

SA/SD includes a variety of notations for formally specifying software, during the analysis phase, data flow diagrams, process specifications, a data dictionary, state transition diagrams and entity-relationship diagrams are used to logically describe a system. In the design phase, details are added to the analysis models and the data flow diagrams are converted into structure chart descriptions of programming language code.

Data flow diagrams model the transformations of data as it flows through a system and are the focus of SA/SD. A data flow diagram consists of processes, data flows, actors, and data stores. Starting from the top level data flow diagram, SA/SD recursively divides complex processes into sub-diagrams, until many small processes are left that are easy to implement. When the resulting processes are simple enough, the decomposition stops, and a process specification is written for each lowest-level process. Process specifications may be expressed with decision tables, pseudocode, or other techniques.

The ***data dictionary*** contains details missing from data flow diagrams. The data dictionary defines data flows and data stores and the meaning of various names.

State transition diagrams model time dependent behavior and are similar to the dynamic model. Most state transition diagrams describe control processes or timing of function execution and data access triggered by events.

Entity Relationship (ER) diagrams highlight relationships between data stores that otherwise would only be seen in the process specifications. Each ER data element corresponds to one data flow diagram data store.

The above tools are used during the process of structured analysis. Structured design follows structured analysis and addresses low-level details. For example, during structured design, data flow diagram processes are grouped into tasks and allocated to operating system processes and CPUs. Data flow diagram processes are converted into programming language functions, and a ***structure chart*** is created showing the procedure call tree.

Merits of SA/SD:

1. SA/SD was one of the first well-thought-out, formal approaches to software and system development.
2. SA/SD is useful for problems where functions are more important and complex than data.

GATEWAY CLASSES

3. An SA/SD design has a clearly-defined system boundary, across which the software procedures must communicate with the real world.
4. Programmers have tended to think in terms of functions, so data flow based methodologies have been easier to learn.

Demerits of SA/SD:

1. SA/SD organizes a system around procedures. In contrast, an object-oriented design technique (such as OMT) organizes a system around real world objects or conceptual objects that exist in the user's view of the world. Most changes in requirements are changes in function rather than in the objects so ***change can be disastrous to procedure-based design***. By contrast, changes in function are readily accommodated in an object-oriented design by adding or changing operations, leaving the basic object structure unchanged,
2. The structure of a SA/SD design is derived in part from the system boundary, so it can be ***difficult to extend*** a SA/SD design to a new boundary. It is much easier to extend an object-oriented design; one merely adds objects and relationships near the boundary to represent objects that existed previously only in the outside world. An object-oriented design is more resilient to change and more extensible.
3. The direct analogy between objects in an object-oriented design and the objects in the problem domain results in systems that are easier to understand. This makes the design more intuitive and simplifies traceability between requirements and software code. It also makes a design more coherent to persons who are not part of the original design team. But SA/SD being oriented towards functions ***lack understandability and traceability*** as it is far from the user's view.
4. In SA/SD the ***decomposition of a process into sub-processes is somewhat arbitrary***. Different people will produce different decompositions. Thus, the ***design components are not reusable*** in this approach. In object-oriented design the decomposition is based on objects in the problem domain; to developers of different programs in the same domain tend to discover similar objects. This increases reusability of components from one project to the next.
5. An object-oriented approach better integrates databases with programming code. One uniform paradigm, the object, can model both database and programming structure. In contrast, a ***procedural design approach is inherently awkward at dealing with databases***. It is difficult to merge programming code organized about functions with a database organized about data.

The benefits of an object oriented approach and the maturation of object-oriented technology will gradually promote its wide use for analysis, design, and implementation.

JACKSON STRUCTURED DEVELOPMENT (JSD)

Jackson Structured Development (JSD) is another mature methodology, which has a different style than SA/SD or OMT. The JSD methodology was developed by Michael Jackson and is especially popular in Europe. JSD first determines the "what" and then the "how". JSD is intended especially for applications in which *timing* is important.

JSD divides system development into three stages:

1. Modeling Stage (entity action step and entity structure step),
2. Network Stage (initial model step, function step, and system timing step)
3. Implementation Stage (implementation step)

Modeling stage

In the modeling stage the designer creates a collection of entity structure diagrams and identifies the entities in the system, the actions they perform, the time-ordering of the actions in the life of the entities, and the attributes of the actions and entities.

Network stage

In the network stage a model of the system as a whole is developed and represented as a system specification diagram (SSD) (also known as a network diagram). Network diagrams show processes and how they communicate with each other, either via state vector connections or via data stream connections. In this stage, the functionality of the system is defined.

Implementation stage

In the implementation stage the abstract network model of the solution is converted into a physical system, represented as a system implementation diagram (SID). The SID shows the system as a scheduler process that calls modules that implement the processes.

GATEWAY CLASSES

JSD uses graphical models as do SA/SD, OMT and other techniques but JSD is *less graphically oriented* than SA/SD and OMT.

A JSD model begins with consideration of the real world. The purpose of a system is to provide functionality but Jackson feels that one must first consider how this functionality fits in with the real world. A JSD model describes the real world in terms of entities, actions and ordering of actions. Entities usually appear as nouns in requirements statements and actions appear as verbs. JSD software development consists of six sequential steps:

1. Entity action step
2. Entity structure step
3. Initial model step

4. Function step
5. System timing step
6. Implementation step

Jackson's elevator example: This example talks about an Elevator Control System which controls two elevators which service six floors. Each elevator has six inside buttons-one for each floor. Each floor has up and down buttons in the waiting area.

1. **Entity Action Step**

During the entity action step the software developer lists entities and actions for part of the real world. The purpose of the overall system guides the choice of entities and actions. The input to the entity action step is the requirements statement; the output is a list of entities and actions.

Jackson identifies two entities for the elevator control example: button and elevator. He identifies three actions: Press a button; elevator arrives at floor n, and elevator leaves floor n.

2. **Entity Structure Step**

Actions occur in ale real world and are not an artifact of the system Actions take place at a point in time, are atomic, and not decomposable. The entity structure step partially orders the actions of each entity by time.

The elevator control system illustrates the importance of ordering actions. It is permissible for an elevator to arrive at floor 3, leave floor 3, arrive at floor 2, leave floor 2, and so on. It does not make sense for two 'arrive' actions to occur in succession; arrive and leave operations must alternate.

3. **Initial Model Step**

The initial model step states how the real world connects to the abstract model. JSD supports state-vector and data stream connection.

The elevator control system illustrates ***state-vector connection***. What happens if someone presses the up button five times rapidly in succession? The elevator user does not want the control system to remember each button press and send an elevator five times to service the request. Instead pressing the up-button sets an "up-flag" to true. Pressing the up button several times has no further effect. The JSD model of the computer system is unaware of the number of presses and only communicates with the real world via the "up-flag". Jackson calls this "up-flag" a state-vector connection.

A computer print buffer illustrates ***data stream connection***. The computer user does not want to lose information if the computer can transmit faster than the printer can print. A print buffer partially decouples the computer from the printer; CPU processing and printing can overlap. A real printer buffer is of finite size; when the buffer is full, the computer must

GATEWAY CLASSES

wait before sending further data. JSD data stream connections are buffers of infinite size. The initial model step of a JSD design, does not address physical buffer limitations.

4. **Function Step**
   The function step uses pseudocode to state outputs of actions. At the end of this step the developer has a complete specification of the required system. In the elevator example, turning the display panel lights on and off as an elevator arrives at each floor is a function that must be specified.

5. **System Timing Step**
   The system timing step considers how much the model is permitted to lag the real world. For the most part, the result of the timing step is a set of informal notes on performance constraints. For example, an elevator control system must detect when up and down buttons are pressed. How long must the user keep the button contact closed upon a press? It is annoying to press an elevator button and not have the system respond.
   The designer explicitly makes performance trade-offs during the system timing step.

6. **Implementation Step**
   The implementation step focuses on the problems of process scheduling and allocates processors to processes. The number of processes may be different from the number of processors. Jackson's elevator control model has 50 processes. The developer must decide whether to match each process to one of 50 CPUs or how to get several processes to share the same CPU. The central concern of implementation step is optimization of the system. It is necessary to reduce the number of processes because it is impossible to provide each process that is contained in specification with its own virtual processor
   After the six JSD steps comes writing of code and database design.

GATEWAY CLASSES

Merits of JSD:

1. JSD is a useful methodology for some specific types of applications such as concurrent software, real time software, microcode and programming parallel computers.
2. JSD was specifically designed to handle difficult real time problems. For these problems, JSD may product a superior design.
3. Jackson places more emphasis on actions and less on attributes, so applies well to applications with such nature.
4. JSD modeling is extremely detailed and focuses on time.

Demerits of JSD:

1. It is not object oriented in pure sense.

2. JSD is complex and difficult to fully comprehend
3. It is less graphically oriented as compared to SA/SD or OMT.
4. JSD is not suitable for the development applications that require high level analysis or have the extensive involvement of databases.

GATEWAY CLASSES

(*)

Mapping object Oriented Concepts using Non-Object Oriented language :- Object Oriented Techniques, the analysis and design can be implemented using a non object oriented language. Though these languages lack some capabilities but it they may benefit from the object oriented analysis & design for sure.

Mapping object oriented concepts using a non object oriented language requires expertise and discipline on the part of programmer because these languages do not have direct features to support object oriented techniques. The programmer needs to use the features of these language and simulate them for the implementation of object oriented concepts :- The steps required for the mapping are as follows :-

(i) ① Translating classes into data structures :- Classes may be implemented as a single contiguous block of attributes such as records of Ada or structures in C. Each attribute of the class becomes an element of the structure that may be of a primitive data type or an array or a structure itself.

(ii) An object must be implemented as a sharable reference permitting the sharing of a single object in memory from multiple possible references

→ Mapping of classes to C structures :- Classes may be mapped to 'C' structs (structures) and an object reference can be represented by a pointer to its structure variable.

Example :-

```
class Student
{
    int rollNo;
    String name;
    float marks;
    --
    --
}
```

Fig: A Java Class.

⇒

```
struct Student
{
    int rollNo;
    char name[20];
    float marks;
};
```

Fig: C structure.

```
struct Student *S1;
```

Fig: Object reference.

② Passing arguments to methods :- Arguments to methods may be simple data values as well as the objects itself. These arguments must preferably be passed by references so that any updation in the value of the object is reflected back to the calling function.

Naming Convention :- It is useful to concatenate classname, two underscores and method name because methods are not part of structures thus

GATEWAY CLASSES

this convention helps in identifying that which method belongs to which structure (class).

⇒ Passing arguements in C :- In C, an object should always be passed by pointer,

```
struct Student s1;  // Object
    Student__setRollNo(*s1, 20);    // Calling of method.
    // Method Definition
    void Student__setRollNo(struct Student *s, int rn)
    {                                   // Object passed by refernce.
        s -> rollNo = rn;
    }
```

As a result rollNo attribute of the s1 object has got the value 20.

③ Implementing inheritance :- Though there is no support for inheritance in non OO languages directly but inheritance may be implemented in the following ways :-

(i) Avoid it :- If applications do not require inheritance then classes not needing inheritance can be implemented as simple structures.

(ii) Flatten the class hierarchy :- Expand each subclass as an independent structure during implementation. Each inherited attribute and operation must be reimplemented as as a seperate attribute and operation on each subclass. Flattening the hierarchy introduces duplication but the constructs like C Macros may limit this duplication.

(iii) Break out seperate objects :- A group of attributes can be pulled out of all the subclasses and implemented as a seperate object with a reference to it stored within each subclass.

⇒ Implementing inheritance in C :- C can implement inheritance in either of the above mentioned ways :-

```
class Student                          struct Student
{                                      {
    int rollNo;                            int rollNo;
    String name;          ⇒            char String name[20];
    ---                                  };
    ---                                  struct ClassRepresentati
}                                      {
class ClassRepresentative extends Student   int rollNo;
{                                           char name[20]
    String represented Course;              char represented
    int represented Sem;                        Course[10]
    ---                                     int represented Se
}                                      };
```

figure : Flattening the class hierarchy

GATEWAY CLASSES

OR

```
struct ClassRepresentative
{
    struct Student *CR;
    char representedCourse[10];
    int representedSem;
};
```

figure: Breaking out seperate object

(4) Implementing Associations :- Association depicts the relationship between two or more classes of objects. Association may be implemented in either of the two following ways :-

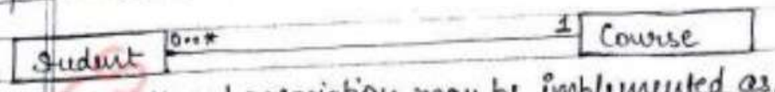
(i) Mapping associations to pointers :- Each object contains a pointer to an associated object or to a set of associated objects. These pointers remains as a field or element of the structure representing class.

(ii) Implementing association objects :- If an association relates more than two classes then it can't be mapped into pointers thus association is implemented as an array or list of the structure variables or structure pointers.

⇒ Implementing association In C :- In C, both the above mentioned approaches can be used for implementing associations.

A binary association is usually implemented as a field in each associated object, containing a pointer to the related object or an array of related objects.

Example :- Student and Course classes,



The above mentioned association may be implemented as,

```
class
struct Student
{
    int rollNo;
    char name[20];
    float marks;
    struct course* cou;
                // single object
};
```

```
struct Course
{
    int courseId;
    char courseDesc;
    struct Student *stu[50];
                // array of-objec pointe
};
```

figure: Implementing bi-directional binary associatio

(5) Implementing Encapsulation :- Encapsulation is one of the major themes of Object Oriented technique and Object Oriented languages provide constructs to implement encapsulation but when the object oriente concepts are mapped to non object oriented language some of the encapsulation is lost but by taking certain precautions, measures and by using language features upto a ce encapsulation can be ensured up to a certain degree.

GATEWAY CLASSES

⇒ Encapsulation in C :- C encourages loose programming style harmful to encapsulation however following steps can be taken to improve encapsulation.

(i) Avoid the use of global variables.

(ii) Package the methods for each class into a seperate file. Only include the declarations for classes whose internal structure you need.

(iii) Donot access the fields of objects of classes, call an access methods only.

(iv) Treat objects of other classes as type "void *".

Using the above described techniques, an object oriented concept can be mapped to a non OO language. But during this the code suffers the loss of

→ Readability & expressive ness
→ Convenience in traversing the class hierarchy
→ Error protection
→ Maintainability

Because these properties are ensured by the language and its constructs in object oriented programming but a non-OO - programmer has to ensure them manually as language has little or no support for these properties.

GATEWAY CLASSES

Object-Oriented Programming Style

As any chess player, cook, or skier can attest, there is a great difference between knowing something and doing it well. Writing programs is no different. It is not enough to know the basic constructs and to be able to assemble them. The experienced programmer follows principles to make readable programs that live beyond the immediate need.

These principles include programming idioms, rules of thumb, tricks of the trade, and cautionary advice. Good style is important in all programming, but it is even more important in Object Oriented (OO) programming, because much of the benefit of the OO approach is predicated on producing reusable, extensible, and understandable programs.

Good programs do more than simply satisfy their functional requirements. Programs that follow proper coding guidelines are more likely to be correct, reusable, extensible, and quickly debugged. Most style guidelines that are intended for conventional programs also apply to OO programs. In addition, facilities such as inheritance are peculiar to OO languages and require new guidelines. We present OO style guidelines under the following categories, although many guidelines contribute to more than one category.

- Reusability
- Extensibility
- Robustness
- Programming-in-the-Large

GATEWAY CLASSES

1. Reusability

Reusable software reduces design, coding and testing cost by amortizing effort over several applications. Reducing the amount of code also simplifies understanding, which increases the likelihood that the code is correct, Reuse is possible in conventional languages, but 00 languages greatly enhance the possibility of code reuse.

1.1 Kinds of Reusability

There are two kinds of reuse:

- sharing of newly written code within an application and
- reuse of previously written code on new applications.

Similar guidelines apply to both kinds of reuse. Sharing of code within an application is a matter of discovering redundant code sequences and using programming-language facilities to consolidate them (refactoring). This kind of code sharing almost always produces smaller programs and faster debugging.

Planning for future reuse takes more foresight and represents an investment. It is unlikely that a class in isolation will be used for multiple applications. Programmers are more likely to reuse carefully thought out subsystems, such as abstract data types, graphics packages, and numerical analysis libraries. Patterns and frameworks can be helpful in this regard.

1.2 Style Rules for Reusability

There are a number of rules that you can follow to promote reusability within your application and across applications.

- ***Keep methods coherent***. A method is coherent if it performs a single function or a group of closely related functions. If it does two or more unrelated things, break it apart into smaller methods.

- ***Keep methods small***. If a method is large, break it into smaller methods. A method that exceeds one or two pages is probably too large. By breaking a method into smaller parts, you may be able to reuse some parts even when the entire method is not reusable.

- ***Keep methods consistent***. Similar methods should use the same names, argument order, data types, return value, and error conditions. Maintain parallel structure when possible. The methods for an operation should have consistent signatures and semantics.

The Unix operating system offers many examples of inconsistent functions. For example, in the C library, there are two inconsistent functions to output strings, puts and fputs. The puts function writes a string to the standard output, followed by a newline character; fputs writes a string to a specified file, without a newline character. Avoid such inconsistency.

- ***Separate policy and implementation***. Policy methods make decisions, shuffle arguments, and gather global context. Policy methods switch control among implementation methods. Policy methods should check for status and errors; they should not directly perform calculations or implement complex algorithms. Policy methods are often application dependent, but they are simple to write and easy to understand. Implementation methods perform specific detailed logic, without deciding whether or why to do them. If implementation methods can encounter errors, they should only return status, not take action. Implementation methods perform specific computations on fully specified arguments and often contain complicated algorithms. Implementation methods do not access global context, make decisions, contain defaults, or switch flow of control. Because implementation methods are self-contained algorithms, they are likely to be meaningful and reusable in other contexts.

GATEWAY CLASSES

Do not combine policy and implementation in a single method. Isolate the core of the algorithm into a distinct, fully specified implementation method. This requires abstracting out the particular parameters of the policy method as arguments in a call to the implementation method.

For example, a method to scale a window by a factor of 2 is a policy method. It should set the target scale factor for the window and call on an implementation method that scales the window by an arbitrary scale factor. If you decide later to change the default scale factor to another value, such as 1.5, you just have to modify the parameter in the policy method, without changing the implementation method that actually does the work.

■ **Provide uniform coverage.** If input conditions can occur in various combinations, write methods for all combinations, not just the ones that you currently need. For example, if you write a method to get the last element of a list, also write one to get the first element. By providing uniform coverage you not only boost reusability, but you also rationalize the scope of related methods.

■ **Broaden the method as much as possible.** Try to generalize argument types, preconditions and constraints, assumptions about how the method works, and the context in which the method operates. Take meaningful actions on empty values, extreme values, and out-of-bounds values. Often a method can be made more general with a slight increase in code.

■ **Avoid global information.** Minimize external references. Referring to a global object imposes required context on the use of a method. Often the information can be passed in as an argument. Otherwise store global information as part of the target object so that other methods can access it uniformly.

■ **Avoid methods with state.** Methods that drastically change behavior depending on previous method history are hard to reuse. Try to replace them with stateless methods. For example, a text-processing application requires insert and replace operations. One approach is to set a flag to *insert* or *replace*, then use *a write* operation to insert or replace text depending on the current flag. A stateless approach uses two operations, *insert* and *replace*, that do the same operations without a state setting. The danger of method states is that an object left in a state in one part of an application can affect a method applied later in the application.

1.3 Using Inheritance

The preceding guidelines improve the chance of sharing code. Sometimes, however, methods on different classes are similar but not similar enough to represent with a single inherited method. There are several techniques of breaking up methods to inherit some code.

■ ***Factor out commonality***. The simplest approach is to factor out the common code into a single method that is called by each method. The common method can be assigned to an ancestor class. This is effectively a subroutine call, as Figure 1.3.1 below shows.

GATEWAY CLASSES

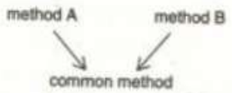


Figure 1.3.1 Code reuse via factoring out commonality. Place the common code into a method that is called by the original methods.

■ ***Factor out differences***. In some cases the best way to increase code reuse between similar classes is to factor out the differences between the methods of different classes, leaving the remainder of the code as a shared method. This approach is effective when the differences between methods are small and the similarities are great. As Figure 1.3.2 shows, the common portion of two methods is made into a new method. The new method calls an operation that is implemented by different methods containing the code differences for each subclass. Sometimes an abstract class must be added to hold the top level method. This approach makes it easier to add new subclasses, because only the difference code need be written.

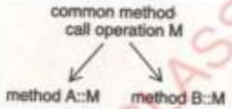


Figure 1.3.2 Code reuse via factoring out differences. Place the difference code in polymorphic methods of a common operation.

GATEWAY CLASSES

A package for plotting numerical data illustrates factoring. DataGraph is an abstract class that organizes common data and operations for its subclasses. One of DataGraph's methods is draw, consisting of the following steps: draw border, scale data, draw axes, plot data, draw title, and draw legend. Subclasses of DataGraph, such as LineGraph, BarGraph, and ScatterGraph, draw borders, titles, and legends the same way but differ in the way they scale data, draw axes, and plot data. Each subclass inherits the methods drawBorder, drawTitle, and drawLegend from abstract class DataGraph, but each subclass defines its own methods for scaleData, drawAxes, and ploiData. The method draw need be defined only once, on class DataGraph, and is inherited by each subclass. Each time the draw method is invoked, it applies drawBorder, drawntle, and drawLegend inherited from the superclass and scaleData, drawAxis, and plotData supplied by the subclass itself. To add a new subclass, only the three specialized methods need be written.

■ ***Delegate***. Sometimes it appears mat use of inheritance would increase code reuse within a program, when a true superclass/subclass relationship does not exist. Do not give in to the temptation to use this implementation inheritance; use delegation instead. Inheritance should be used only when the generalization relationship is semantically valid. Inheritance means that each instance of a subclass truly is an instance of the superclass; thus all operations and attributes of the superclass must uniformly apply to the subclasses. Improper use of inheritance leads to programs that are hard to maintain and extend.

00 languages are permissive in their use of inheritance and will not enforce the good programming practice that we recommend.

Delegation provides a proper mechanism to achieve the desired code reuse. The method is caught in the desired class and forwarded to another class for actual execution. Since each method must be explicitly forwarded, unexpected side effects are less likely to occur. The names of methods in the catching class may differ from those in the supplier class. Each class should choose names that are most appropriate for its purposes.

■ ***Encapsulate external code***. Often you will want to reuse code that may have been developed for an application with different interfacing conventions. Rather than inserting a direct call to the external code, it is safer to encapsulate its behavior within a method or a class. This way, the external routine or package can be changed or replaced, and you will have to change your code in only one place.

For example, you may have a numerical analysis application but, knowing that reliable matrix-inversion software already exists, you do not want to re-implement the algorithm. You could write a matrix class to encapsulate the functionality provided by the external subroutine package. The matrix class would have, for example, an inverse method that takes the tolerance-for-singularity as an argument and returns a new matrix that is the inverse of the method's target.

GATEWAY CLASSES

2. Extensibility

Most software is extended in ways that its original developers never expect. The reusability guidelines enhance extensibility, as do the additional guidelines listed below.

■ ***Encapsulate classes***. A class is encapsulated if its internal structure is hidden from other classes. Only methods on the class should access its implementation. Many compilers are smart enough to optimize methods into direct access to the implementation, but the programmer should not. Respect the information in other classes by never reaching inside the class for data.

■ ***Specify visibility for methods***. Public methods are visible outside a class and have published interfaces. Once a public method is used by other classes, it is costly to change its interface, so carefully define and limit the number of public methods. In contrast, private methods are internal to a class—they can be deleted or changed with impact limited to other methods on the class. Protected methods have intermediate visibility.

Careful use of visibility makes your classes easier to understand and increases code resilience. Private and protected methods suppress unnecessary details from the user of a class, avoiding confusion. Unlike a public method, a private method cannot be applied out of context, so it can rely on preconditions or state information of the class.

■ ***Hide data structures***. Do not export data structures from a method. Internal data structures are specific to a method's algorithm. If you export them, you limit flexibility to change the algorithm later.

■ ***Avoid traversing multiple links or methods***. A method should have limited knowledge of a class model. A method must be able to traverse links to obtain its neighbours and must be able to call methods on them, but it should not traverse a second link from the neighbor to a third class because the second link is not directly visible to it. Instead, call a method on the neighbor object to traverse nonconnected objects; if the association network changes, the method can be rewritten without changing the call.

Similarly, avoid applying a second method to the result of a method call unless the class of the result is already known as an attribute, argument, or neighbor, or the result class is from a lower-level library. Instead, write a new method on the original target class to perform the combined method itself. The principles in this bullet were proposed as the "Law of Demeter."

■ ***Avoid case statements on object type***. Use polymorphism instead. Case statements can be used to test internal attributes of an object but should not be used to select behaviour based on object type. The dispatching of operations based on object type is the whole point of polymorphism, so don't circumvent it.

■ ***Design for portability***. Limit system dependencies to a few basic methods. You can then more easily port software to other hardware platforms and software environments.

For example, if you are using a database, you might isolate database access. This would make it easier to switch database vendors and possibly enable switching paradigm (such as from relational to OO database).

GATEWAY CLASSES

3. Robustness

You should strive for efficient methods but not at the expense of robustness. A method is robust if it does not fail even if it receives improper parameters. In particular, you should never sacrifice robustness against user errors.

■ ***Protect against errors***. Software should protect itself against incorrect user input and never let it cause a crash. Methods must validate input that could cause trouble.

There are two kinds of errors. Analysis uncovers errors that exist in the problem domain and determines an appropriate response. For example, an ATM should handle errors for the card scanner and communications lines. On the other hand, low-level system errors concern programming aspects. These low-level errors include memory allocation errors, file input/output errors, and hardware faults. Your program should check for system errors and at least try to die gracefully if nothing else is possible.

Try to guard against programming bugs as well, and give good diagnostic information. During development, it is often worthwhile to insert internal assertions into the code to uncover bugs, even though the checks will be removed for efficiency in the production version. A strongly typed 00 language provides greater protection against type mismatches, but you can insert assertions in any language. In particular, you should check array bounds.

■ ***Optimize after the program runs***. Don't optimize a program until you get it working. Many programmers spend too much time improving code that seldom executes. Measure the performance within the program first; you may be surprised to find that most parts consume little time. Study your application to learn what measures are important, such as worst-case times and method frequencies. If a method may be implemented in more than one way, assess the alternatives with regard to memory, speed, and implementation simplicity. In general, avoid optimizing more of the program than you have to, since optimization compromises extensibility, reusability, and understandability. If methods are properly encapsulated, you can replace them with optimized versions without affecting the rest of the program.

■ ***Always construct objects in a valid state***. Otherwise, you create a situation where someone might use the constructor and not call the right subsequent method. As a matter of good software practice, your code should always be logically sound.

■ ***Validate arguments***. Public methods, those available to users of the class, must rigorously check their arguments, because users may violate restrictions. Private and protected methods can improve efficiency by assuming that their arguments are valid, since the implementor can rely on the public methods that call them for error checking.

Don't write or use methods whose arguments can't be validated. For example, the infamous scanf function in Unix reads a line of input into an internal buffer without checking the size of the buffer. This loophole has been exploited to write virus programs that force a buffer overflow in system software that did not validate its arguments.

■ ***Avoid predefined limits***. When possible use dynamic memory allocation to create data structures without predefined limits. It is difficult to predict the maximum capacity expected of data structures in an application, so don't set any limits. The day of fixed limits on symbol table entries, user names, file names, compiler entries, and other things should be long over. Most OO languages have excellent dynamic memory allocation facilities.

■ ***Instrument the program for debugging and performance monitoring***. Just as a hardware circuit designer instruments an IC board with test points, you should instrument your code for testing, debugging, statistics, and performance. The level of debugging that you must build into your code depends on your language's programming environment. You can add debug statements to methods that conditionally execute depending on the debug level. The debug statements print a message on entry or exit and selective input and output values.

You can better understand the behavior of classes by adding code to gather statistics. Some operating systems, such as Unix, offer tools to create execution profiles of an application. Typically, these tools report the number of times each method was called and the amount of processor time spent in each method. If your system lacks comparable tools, you can instrument your code for gathering statistics much like for debugging.

GREENWAY CLASSES

4. Programming-in-the-Large

Programming-in-the-large refers to writing large, complex programs with teams of programmers. Human communication becomes paramount on such projects and requires proper software engineering practices. You should observe the following guidelines.

■ ***Do not program prematurely***. Think long and carefully before you commit to code. Ultimately, of course you must have code to deliver your application. But code is tedious to write and difficult to change. In contrast, models are much more malleable, because they are high level and suppress details. It is much better to work out your ideas with models, gain a full understanding, and only then write the code.

■ ***Make methods readable***. Meaningful variable names increase readability. Typing a few extra characters costs less than the misunderstanding that can come later when another programmer tries to decipher your variable names. Avoid confusing abbreviations, and use temporary variables instead of deeply nested expressions. Do not use the same temporary variable for two different purposes within a method, even if their usage does not overlap; most compilers will optimize this anyway. Minor improvements to efficiency are not worth compromising readability.

■ ***Keep methods understandable***. A method is understandable if someone other than the creator can understand the code (as well as the creator after a time lapse). Keeping methods small and coherent helps to accomplish this.

■ ***Use the same names as in the class model***. The names used within a program should match those in the class model. A program may need to introduce additional names for implementation reasons, but you should preserve the names for concepts that carry forward. This practice improves traceability, documentation, and understandability. It is reasonable to adopt conventions such as uniform prefixes for consistency across applications, thereby avoiding name clashes.

■ ***Choose names carefully***. Make sure that your names accurately describe the operations, classes, and attributes that they label. Follow a uniform style in devising names.

For example, you may name operations as "verbObject," such as addElement or drawHighlight. Be sure to define a vocabulary of verbs that are often used. For example, don't use both new and create unless they have different meanings. Many 00 languages automatically build method names from the class and operation names.

Similarly, do not use the same method name for semantically different purposes. As Figure 4.1 shows, all methods with the same name should have the same signature (number and types of arguments) and meaning.

Circle::area() Rectangle::area()	~~Matrix::invert()~~ — performs matrix inversion ~~Figure::invert()~~ — turns figure upside down
Correct	*Try to avoid*

Figure 4.1 Method names. All methods with the same name should have the same signature and meaning

GATEWAY CLASSES

■ ***Use programming guidelines***. Project teams should use programming guidelines available in their organizations or external guidelines. Guidelines address issues such as the form of variable names, indentation style for control structures, method documentation headers, and in-line documentation.

■ ***Use packages***. Group closely related classes into a package.

■ ***Document classes and methods***. The documentation of a method describes its purpose, function, context, inputs, and outputs as well as any assumptions and preconditions about the state of the object. You should describe not only the details of an algorithm, but also why it was chosen. Internal comments within the method should describe major steps.

■ ***Publish the specification***. The specification is a contract between the producer and the consumer of a class. Once a specification is written, the producer cannot break the contract, for doing so would affect the consumer. The specification contains only declarations, and the user should be able to use the class just by looking at the declarations. Online descriptions of a class and its features help promote the correct use of the class. Below mentioned is a class specification:

GATEWAY CLASSES

Class name: Circle

Version: 1.0

Description: Ellipse whose major and minor axes are equal

Super classes: Ellipse

Features:

Private attributes:

center: Point — location of its center

radius: Real — its radius

Public methods:

draw (Window) — draws a circle in the window

intersectLine (Line): Set of Points — finds the intersection of a line and a circle, returns set of 0-2 points

area(): Real — calculates area of circle

perimeter(): Real — calculates circumference of circle

Private methods: none

Procedural vs. Object Oriented Programming :-

Object oriented programming is mainly involves two key concepts, objects and classes whereas procedural programming stresses on procedures or functions for doing the work. Both are two different programming paradigm having their own pros & cons but the striking features of OOP may overpower the procedural approach as it is gaining popularity world wide.

The difference between these two approaches may be summarized through the following table :-

Point of Distinction	Procedural Programming	Object Oriented Programming (OOP
① Fundamental Idea	Program is divided into functions or procedures that has a well defined purpose and interface and is composed of a list of instructions that tells computer to do something.	OOP combines both data and functions that operate on that data in a single unit named Class and Objects are the instances of the classes.
② Value of data.	Data is undervalued and given least importance.	Data is given prime importance. It is encapsulated and hidden from the outside world.

GATEWAY CLASSES

Point of Distinction	Procedural Programming	Object Oriented Programming
(3) Relationship to the real world	Functions and datastructure do not model the real world well.	Close match between programming objects + real world Objects.
(4) Decomposition approach	Algorithmic	Object Oriented
(5) Software engineering Approach	Top-down	Bottom up.
(6) Encapsulation	No direct provision for Encapsulation	OOP has specific encapsulation constructs such as classes.
(7) Inheritance	Not supported	One of the major themes of OOP
(8) Code sharing + reuse	Limited to the functions	Inheritance leads to codesharing + reuse.
(9) Polymorphism	Not directly supported	Polymorphism may be achieved through method overloading, overriding, dynamic method dispatch etc.
(10) Ease of extension + maintenance.	Procedural programs are not usually extensible and more complex to write and maintain	OOP leads to extensible programs that are easy to read, write + maintain.

GATEWAY CLASSES

Object Oriented Language Features

Object oriented languages vary in their support of advanced object oriented concepts. There is no single language that matches all styles and meets all needs. Here, we discuss various features of object oriented languages that vary among languages. We have to carefully select a language that best suits to our requirements. The features are summarized as below:

1. Inheritance

Inheritance allows one object to acquire the properties and behavior of another object. The striking advantage of inheritance is *reusability*. Though, being an important object oriented concept, inheritance is supported across object oriented languages. The point where the languages vary is the support for one of controversial type of inheritance i.e. ***Multiple Inheritance*** as it introduces the possibility of conflict among attribute and operation names. A class could have more than one ancestor with the same attribute or operation. Various languages have their own way of handling with this problem for example C++ handles this situation with the help of virtual functions but supports multiple inheritance where as Java completely rules it out and does not support multiple inheritance.

2. Class Library

Most Object Oriented languages include a library of useful generic classes, which may be used as is or customized to meet a specific need through the creation of subclasses. The availability of a class library means that many components need not be re-implemented by the programmer. For example, the *collections framework* of Java, here the most useful classes implement general purpose data structures, such as dynamic arrays, lists, sets, and maps etc.

3. Strong Typing versus Weak Typing

The term *typing* refers to whether each variable and attribute value is merely known to be an object (*weak typing*) or may be declared more precisely as belonging to a particular class or one of its subclasses (*strong typing*).Smalltalk is a weakly typed languages whereas languages such as C++, Java are strongly typed. C++, Java and some other languages also allows an attribute to be of a predefined data type that is considered to be non-object. Permitting these non-object values increases efficiency of common operations such as arithmetic operations.

The advantage of strong typing is that it is easier to detect and correct software bugs at compile time rather than at run-time. Strong typing improves the reliability of delivered software. Due to this, modern programming languages have moved in the direction of strong typing. Even the power of weak typing may also be gained in strong typing as the programmer can always declare everything to be of class "Object" to get the same effect as weak typing. Java has a predefined class named Object that is implicitly defined to be superclass of every class in Java.

GATE CS CLASSES

4. Memory Management

Most OO languages allocate memory from a heap rather than using fixed blocks of global memory or a stack. A dynamic memory allocation system can run out of memory unless objects that are no longer needed are deallocated.

There are two approaches to memory management (or deallocating unused memory):

i. ***Automatically by the run-time system of the languages.***

This approach relieves the programmer of the responsibility of deciding when to deallocate memory but may result in long pauses at unpredictable points during execution (this may be avoided through incremental garbage collection but it imposes higher average cost).

Java supports this approach as it offers automatic garbage collection feature.

ii. ***Explicitly by deallocation statements written by the programmer.***

Explicit deallocation runs the dual risk of deallocating objects that are still referenced (dangling pointers leading to program crashes) and failing to deallocate objects that are inaccessible (leading to memory wastage), but it does not impose any unpredictable additional time or memory costs on the program.

C++ requires the programmer to deallocate unneeded objects, but the C++ language allows the programmer to define destructor functions, which are automatically called when an object goes out of scope. It is not a garbage collector but it gives a convenient way of organizing and invoking explicit storage deallocation using the delete operator.

GATEWAY CLASSES

5. Encapsulation

Encapsulation (based on information hiding) makes us able to *combine the data structure and behavior in a single entity* called class. It consists of separating the external aspects of an object that are accessible to other objects, from the internal implementation details of the object, which are hidden from other objects. Encapsulation prevents a program from becoming so interlaced that a small change has massive ripple (side) effects. We must be able to change the implementation of a class without affecting its users.

Direct access to the attributes of a class of objects must be strictly forbidden. The proper way of accessing attribute of another object is to "*ask for it*" by invoking an operation of the object, rather than simply "*taking it*". We have the concept of *access specifiers* (public, private and protected) to control the access of class elements. C++ by default considers all the class elements to be private where as Java considers them accessible all over in the package. C++ *friend* declaration allows limited loosening of encapsulation for access by specified classes or

functions. Encapsulation can be violated through *careless use of inheritance* also. Methods of the subclass must never access the attributes of the super class directly. C++ permits a class to restrict its visibility from subclasses.

6. Packaging

A class is a structuring construct but is not adequate from large systems. So for large systems, we require packages. A package is a structuring mechanism that contains classes, interfaces and other packages. Packages offer two advantages: *namespace resolution* (allowing two elements to have same name in different packages) and *visibility control* (controlling access of class and its elements). C++ supports packaging through the concept of namespaces and Java has package as a language construct itself.

7. Development Environment

The tools such as *Integrated Development Environment (IDE)*, available for browsing and editing source code, compilation, testing & debugging, and system integration have a big effect on developer's productivity. These tools are especially important in object oriented languages because of the difficulty of managing inherited and dynamically bound features in a large system. The IDEs such as *Eclipse* offers enormous features to aid the development work, though it is used primarily for Java development but its extensive set of plug-ins support development in other object oriented languages as well.

GATEWAY CLASSES

8. Metadata

Metadata is data about data. Metadata that is present at run-time allows an application to reason about, and possibly even change its own structure and capabilities, including the operations an object supports, the attributes it possesses, or the types of attributes.

9. Parameterized Classes

Parameterized Classes, or generics, allow a parameterized template to be written that can be applied in several cases that differ only in the types of the parameters. For example, a generic '*List*' class can be instantiated as a list of integers, a list of strings or a list of user defined objects. C++ supports parameterized classes through *templates* and Java supports it through *Generics*.

10. Data Persistence

All computer programs operate on data. If we require data that persists beyond the lifetime of a single program execution, then you will need to use a permanent data store. There are various reasons for having persistent data:

- Persistent data may provide the easiest mechanism for passing data from one program to another.
- Persistent data allows the same program to resume processing at a later date.
- Data stores are often useful for historical or archival purposes.

Object oriented languages support data persistence by supporting files through *file handling* features and by supporting databases through *database handling*. Java has *Java Database Connectivity (JDBC)* API to support database handling.

GATEWAY CLASSES

Thank you

GATEWAY CLASSES